

MongoDB

Conceitos Fundamentais

JSON (JavaScript Object Notation)

- É uma formatação leve para troca de dados, fácil de ler para humanos e fácil de interpretar para máquinas.
 - **Estrutura:** Coleção de pares nome/valor e listas ordenadas de valores.

```
2 {
3   "name": "Foo",
4   "age": 15,
5   "height": 1.78,
6   "accountMoney": 1.7e8,
7   "male": true,
8   "items": ["keys", "Glasses"],
9   "address": {
10    "city": "San Francisco",
11    "state": "CA"
12  }
13 }
```

MongoDB e seus Componentes

Definição: Banco de dados open-source, multiplataforma, escrito em C++, que utiliza o motor V8 do Google para interpretar JavaScript. O nome vem de "humongous" (gigantesco).

Estrutura de Armazenamento:

- **DB:** Base de dados.
- **Coleção:** Similar a uma "tabela" em bancos relacionais, é um conjunto de documentos.
- **Documento:** É cada registro salvo (a linha da tabela).
- Apresenta campos dinâmicos
 - Não requer que todos documentos compartilhem a mesma estrutura

Operações CRUD e Buscas

Insert

- `db.colecao.insertOne({...})`

```
db.inventory.insertMany([
  { item: "journal", qty: 25, status: "A", size: { h: 14, w: 21, uom: "cm" }, tags: [ "blank", "red" ] },
  { item: "notebook", qty: 50, status: "A", size: { h: 8.5, w: 11, uom: "in" }, tags: [ "red", "blank" ] },
  { item: "paper", qty: 100, status: "D", size: { h: 8.5, w: 11, uom: "in" }, tags: [ "red", "blank", "plain" ] },
  { item: "planner", qty: 75, status: "D", size: { h: 22.85, w: 30, uom: "cm" }, tags: [ "blank", "red" ] },
  { item: "postcard", qty: 45, status: "A", size: { h: 10, w: 15.25, uom: "cm" }, tags: [ "blue" ] }
]);
```

- `db.colecao.insertMany([...])`

```
db.inventory.insertOne({ item: "book", qty: 5, status: "A", size: { h: 9.5, w: 10, uom: "in" }, tags: [ "green", "blank" ] })
```

UPDATE

- `db.colecao.update(filtro, modificação)`
- **\$set**: Altera ou cria um valor específico. Ex: `{ $set: {preco: 12} }`.
- **\$inc**: Incrementa um valor numérico. Ex: `{ $inc: {preco: 5} }` (aumenta o preço em 5).

```
db.produtos.update(  
  {nome: "camisa"},  
  { $set: {preco: 12} }  
);
```

```
db.produtos.update(  
  {nome: "calca"},  
  { $inc: {preco: 5} }  
);
```

DELETE

- `db.colecao.remove(filtro)`.
- Para remover apenas o primeiro item encontrado, usa-se o parâmetro `true`:
 - `db.produtos.remove({nome: "bone"}, true)`.

```
db.produtos.remove({nome: "luva"});  
  
// Remove apenas 1  
db.produtos.remove({nome: "bone"}, true);
```

Seleção

- Buscar todos documentos:
 - `db.inventory.find({})`
- Buscar documento por um valor (exato):
 - `db.inventory.find({ status: "D" })`
- Buscar por níveis diferentes (exato):
 - `db.inventory.find({ size: { h: 14, w: 21, uom: "cm" } })`
- Buscar baseada em elementos de listas:
 - `db.inventory.find({ tags: "red" })`

```

db.produtos.find();

db.produtos.find().sort({nome:1});

db.produtos.find({preco: 20});

db.produtos.find({preco:{$gt: 15}});

db.produtos.find({tamanho:{$in: ["p", "m"]}});

db.produtos.find({$or: [{tamanho: "p"}, {tamanho: "m"}]});

db.produtos.find({tamanho: {$exists: false}});

```

Filtros

Operadores de Comparação:

- `$gt` (maior que), `$gte` (maior ou igual).
- `$lt` (menor que), `$lte` (menor ou igual).
- `$ne` (diferente).

Operadores Lógicos e Outros:

- `$or`, `$and`, `$not`, `$nor`.
 - `$in` (dentro de um array), `$exists` (verifica se atributo existe).
 - `$nin` (não dentro do array)
 - `$all` (todos do array)
 - `$type` (o dado é do tipo)
-
- Múltiplas propriedades são implicitamente conjunções (AND):
 - `db.inventory.find( { status: "A", qty: { $lt: 30 } } )`
 - Operador OR:
 - `db.inventory.find( { $or: [ { status: "A" }, { qty: { $lt: 30 } } ] } )`
 - Documentos embarcados:
 - `db.inventory.find( { "size.h": { $lt: 15 } } )`
 - Busca em lista (exato em mesma ordem e exato em qualquer ordem)
 - `db.inventory.find( { tags: ["red", "blank"] } )`
 - `db.inventory.find( { tags: { $all: ["red", "blank"] } } )`

Projeção

- Campos a serem projetados [coleção | filtro | projeção]:

- `db.inventory.find({ status: "A" }, { item: 1, status: 1 })`
- Ignorar campos:
 - `db.inventory.find({ status: "A" }, { item: 1, status: 1, _id: 0 })`
- Projetar campos de documentos embarcados:
 - `db.inventory.find({ status: "A" }, { item: 1, status: 1, "size.uom": 1 })`
- Projetar última posição de arrays:
 - `db.inventory.find({ status: "A" }, { item: 1, status: 1, "tags": { $slice: -1 } })`

Índices

- Um atributo ascendente**
`db.produtos.ensureIndex({nome: 1});`
- Um atributo descendente**
`db.produtos.ensureIndex({nome: -1});`
- Um atributo único:** (Evita que existam dois produtos com o mesmo nome)
`db.produtos.ensureIndex({nome: 1}, {unique: true});`
- Remove:** (Apaga o índice criado anteriormente)
`db.produtos.dropIndex({nome: 1});`

Característica	Documento Embarcado {}	Array []
O que é?	Um objeto com propriedades nomeadas.	Uma lista de valores ordenados.
Símbolo	Chaves <code>{ ... }</code>	Colchetes <code>[...]</code>
Exemplo	<code>size: { h: 10, w: 20 }</code>	<code>tags: ["vermelho", "azul"]</code>
Lógica	"O produto tem um tamanho detalhado"	"O produto tem várias tags"
Busca	<code>find({ "size.h": 10 })</code>	<code>find({ tags: "vermelho" })</code>

Cursors

Iterar por um cursor

```
var myCursor = db.inventory.find({});
while (myCursor.hasNext()) {
  print(tojson(myCursor.next()));
}
```

Obter array de um cursor

```
var myCursor = db.inventory.find({});
var documentArray = myCursor.toArray();
var myDocument = documentArray[3];
printjson(myDocument);
```

Buscas textuais

- Dados:

```
db.stores.insert([
  { _id: 1, name: "Java Hut", description: "Coffee and cakes" },
  { _id: 2, name: "Burger Buns", description: "Gourmet hamburgers" },
  { _id: 3, name: "Coffee Shop", description: "Just coffee" },
  { _id: 4, name: "Clothes Clothes Clothes", description: "Discount clothing" },
  { _id: 5, name: "Java Shopping", description: "Indonesian goods" }
])
```
- Necessário um índice para o campo:
 - `db.stores.createIndex( { name: "text", description: "text" } )`
- Ou lógico por palavras tokenizando os textos por espaço e pontuações:
 - `db.stores.find( { $text: { $search: "java coffee shop" } } )`
- Excluir termo:
 - `db.stores.find( { $text: { $search: "java shop -coffee" } } )`

Buscas Geoespaciais

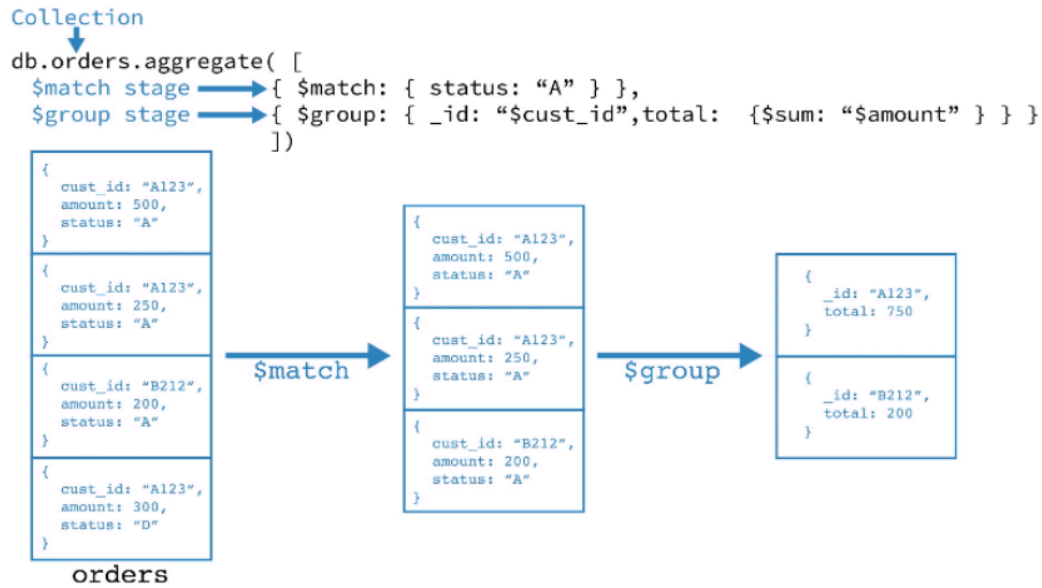
- Formato GeoJSON: `<field>: { type: <GeoJSON type> , coordinates: <coordinates> }`

```
db.places.insert({name:"Central Park",loc:{type:"Point",coordinates:[-73.97,40.77]},category:"Parks"});
db.places.insert({name:"Roosevelt Park",loc:{type:"Point",coordinates:[-73.9928,40.7193]},category:"Parks"});
db.places.insert({name:"Polo Grounds",loc:{type:"Point",coordinates:[-73.9375,40.8303]},category:"Stad."});
```
- Criar um índice de suporte das buscas geoespaciais:
 - `db.places.createIndex({ loc: "2dsphere" })`
- Buscar pontos ao menos 1000 e no máximo 5000 metros de distância:

```
db.places.find({ loc: {
  $near: {
    $geometry: { type: "Point", coordinates: [ -73.9667, 40.78 ] },
    $minDistance: 1000,
    $maxDistance: 5000
  }
}})
```

Agregação

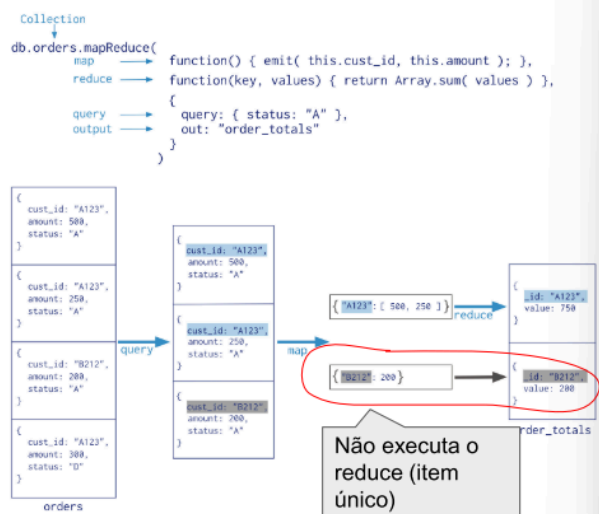
- Processa dados em estágios (Pipeline).
- **\$match**: Filtra os documentos (similar ao WHERE).
- **\$group**: Agrupa e sumariza dados (ex: `total: { $sum: "$amount" }`).



Map-reduce

- **Map:** Função que emite pares chave/valor (emit).
- **Reduce:** Função que processa/soma os valores emitidos para cada chave.

- Função map - agrupa os valores do campo "amount" por ID;
- Reduce -> sumariza os valores para cada entrada (somente para entradas com mais de um item);



Manipulação e Consulta de Dados

Modelo de Dados (Cenário da Loja Online)

Coleção: Usuários

Estrutura do Documento

A coleção `usuarios` armazena informações cadastrais dos clientes da loja online.

Campos principais		
<code>_id</code>	ObjectId	Identificador único
<code>nome</code>	String	Nome completo
<code>email</code>	String	E-mail para contato
<code>endereco</code>	Object	Dados de endereço
<code>data_cadastro</code>	ISODate	Data de registro

Exemplo de Documento

```
{
  "_id": ObjectId("5f8d2a2b9d3b6c0b7c9d5e4f"),
  "nome": "Maria Silva",
  "email": "maria@exemplo.com",
  "telefone": "11999990000",
  "endereco": {
    "rua": "Av. Principal, 123",
    "cidade": "São Paulo",
    "estado": "SP",
    "cep": "01001-000"
  },
  "data_cadastro": ISODate("2023-01-15T10:30:00Z"),
  "preferencias": ["Eletrônicos", "Livros"]
}
```

- Observações**
- Usa documento aninhado para `endereco`
 - Armazena preferências em um array
 - Campo `email` deve ser único (índice)

Coleção: Produtos

Estrutura do Documento

A coleção `produtos` armazena o catálogo de itens disponíveis na loja online.

Campos principais		
<code>_id</code>	ObjectId	Identificador único
<code>nome</code>	String	Nome do produto
<code>categoria</code>	String	Categoria principal
<code>preco</code>	Number	Valor em reais
<code>estoque</code>	Number	Quantidade disponível
<code>tags</code>	Array	Palavras-chave

Exemplo de Documento

```
{
  "_id": ObjectId("6f8e3b4c5a2d9e7f0b1c3d5e"),
  "nome": "Notebook UltraBook X5",
  "categoria": "Eletrônicos",
  "preco": 4999.90,
  "estoque": 15,
  "descricao": "Notebook com processador i7, 16GB RAM",
  "tags": ["notebook", "computador", "ultrabook"],
  "data_cadastro": ISODate("2023-03-10T08:45:12Z"),
  "fabricante": "TechBrand",
  "especificacoes": {
    "processador": "Intel Core i7",
    "memoria": "16GB DDR4",
    "armazenamento": "SSD 512GB"
  }
}
```

Observação: Esta coleção demonstra recursos do MongoDB como documentos aninhados (`especificacoes`) e arrays (`tags`), permitindo estrutura flexível de dados.

Coleção: Pedidos

Estrutura do Documento

A coleção `pedidos` representa as transações de compras realizadas pelos usuários na loja online.

Campos principais		
<code>_id</code>	ObjectId	Identificador único
<code>usuario_id</code>	ObjectId	Referência ao usuário
<code>itens</code>	Array	Produtos comprados
<code>valor_total</code>	Number	Valor do pedido
<code>status</code>	String	Situação atual

Relacionamentos

`pedidos.usuario_id` → `usuarios_id`
`pedidos.itens.produto_id` → `produtos_id`

Exemplo de Documento

```
// Estrutura básica de um pedido
{
  "_id": ObjectId("7a8b9c0d1e2f3g4h5i6j7k8l"),
  "usuario_id": ObjectId("5f8d2a2b9d3b6c0b7c9d5e4f"),
  "data": ISODate("2023-06-15T14:30:45Z"),
  "itens": [
    {
      "produto_id": ObjectId("6f8e3b4c5a2d9e7f0b1c3d5e"),
      "nome": "Notebook UltraBook X5",
      "quantidade": 1,
      "preco": 4999.90
    }
  ],
  "valor_total": 4999.90,
  "status": "entregue"
}
```

Observação: Os itens armazenam informações básicas dos produtos no momento da compra, permitindo manter o histórico mesmo que o produto seja alterado posteriormente.

Comando insertOne: Conceito e Exemplo

Conceito e Sintaxe

O método `insertOne()` insere um único documento em uma coleção. É a operação mais básica de inserção no MongoDB.

Sintaxe

```
db.collection.insertOne(document)
```

Retorno

```
{ acknowledged: true, insertedId: ObjectId("...") }
```

Exemplo Prático

```
// Inserindo um usuário na coleção
// Inserir um novo usuário
db.usuarios.insertOne({
  "nome": "Carlos Oliveira",
  "email": "carlos@exemplo.com",
  "telefone": "31987654321",
  "endereco": {
    "rua": "Av. Central, 789",
    "cidade": "Belo Horizonte",
    "estado": "MG",
    "cep": "30100-000"
  },
  "data_cadastro": new Date()
})

// Resultado:
// { acknowledged: true, insertedId: ObjectId("6487a82d12e43a9e5f9123ef") }
```

Comando insertMany: Inserindo Vários Documentos

O que é insertMany?

O comando `insertMany()` permite inserir múltiplos documentos em uma coleção em uma única operação, oferecendo melhor performance que inserções individuais.

Sintaxe Básica

```
db.collection.insertMany(
  [ {documento1}, {documento2}, ... ],
  { opcoes }
)
```

Principais características

- **Operação em lote:** Insere múltiplos documentos com uma única chamada ao servidor
- **Comportamento atômico:** Por padrão, se um documento falhar, os outros continuam sendo inseridos (ordered: false)

Exemplo Prático

```
// Inserindo múltiplos produtos
db.produtos.insertMany([
  {
    nome: "Smartphone Galaxy S21",
    categoria: "Eletrônicos",
    preco: 3999.90,
    estoque: 25,
    tags: ["smartphone", "samsung", "android"]
  },
  {
    nome: "Fone de Ouvido Bluetooth",
    categoria: "Acessórios",
    preco: 199.90,
    estoque: 50,
    tags: ["fone", "bluetooth", "audio"]
  }
])

// Resultado retornado:
{
  "acknowledged": true,
  "insertedIds": {
    "0": ObjectId("6a8f2c3d4e5f6g7h8i9j0k1l"),
    "1": ObjectId("7b1c2d3e4f5g6h7i8j9k0l1m")
  }
}
```

Comando updateOne: Conceito e Sintaxe

Atualização de Documentos

O comando `updateOne()` é utilizado para modificar um único documento que corresponda a um critério de filtro especificado.

Sintaxe Básica

```
db.collection.updateOne(
  <filtro>,
  <atualização>
)
```

Principais Operadores

\$set	Substitui o valor de um campo
\$inc	Incrementa o valor de um campo numérico
\$push	Adiciona um elemento a um array

Exemplo Prático

```
// Atualizar informações de um usuário
// Atualizar endereço de um usuário
db.usuarios.updateOne(
  { email: "maria@exemplo.com" },
  {
    $set: {
      "telefone": "11988887777",
      "endereco.cidade": "Campinas",
      "endereco.estado": "SP"
    }
  }
)

// Resultado:
// { "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }
```

Dicas Importantes

- **Atomicidade:** A operação é atômica
- **Filtros precisos:** Use filtros específicos para garantir que apenas o documento correto seja atualizado

Comando updateMany e Operadores de Atualização

Atualizações em Múltiplos Documentos

`updateMany()` modifica vários documentos que correspondem ao filtro especificado, enquanto `updateOne()` atualiza apenas o primeiro documento encontrado.

```
db.collection.updateMany(
  <filtro>,
  <atualização>,
  { upsert: <boolean> }
)
```

Principais Operadores

\$set Define valor de um campo	\$inc Incrementa valor numérico	\$push Adiciona item a um array	\$pull Remove itens de array
--	---	---	--

Cuidado: Sempre verifique seus filtros antes de usar `updateMany`.
Filtragem incorreta pode atualizar documentos indesejados.

Exemplos Práticos

```
// Atualizar categoria de produtos
db.produtos.updateMany(
  { categoria: "Eletrônicos" },
  { $set: {
    categoria_principal: "Tecnologia",
    ultima_atualizacao: new Date()
  } }
)

// Resultado: { "matchedCount": 15, "modifiedCount": 15 }
```

```
// Incrementar estoque e adicionar tags
// Incrementar estoque de produtos com poucos itens
db.produtos.updateMany(
  { estoque: { $lt: 5 } },
  {
    $inc: { estoque: 10 },
    $push: { tags: "reabastecido" }
  }
)
```

- `updateOne()` (atualiza o primeiro encontrado) e `updateMany()` (atualiza todos que batem com o filtro).
- Operadores Principais:**
 - \$set** : Substitui ou define o valor de um campo.
 - \$inc** : Incrementa um valor numérico (ex: aumentar estoque).
 - \$push** : Adiciona um item a um array.
 - \$pull** : Remove itens de um array.`db.cliente.find({}, { nome: 1, endereco: 1, _id: 0 })`

Comandos Delete: deleteOne e deleteMany

Removendo Documentos

Os comandos `deleteOne()` e `deleteMany()` permitem remover documentos de uma coleção de acordo com filtros específicos.

Sintaxe Básica

```
db.collection.deleteOne(
  <filtro>
)

db.collection.deleteMany(
  <filtro>
)
```

Diferenças Principais

Comando	Comportamento
<code>deleteOne()</code>	Remove apenas o primeiro documento encontrado
<code>deleteMany()</code>	Remove todos os documentos que correspondem ao filtro

Exemplo Prático

```
// Exemplos de uso

// Remove um produto específico pelo nome
db.produtos.deleteOne(
  { nome: "Teclado Mecânico Gamer" }
)

// Resultado:
// { "acknowledged" : true, "deletedCount" : 1 }

// Remove todos os produtos sem estoque
db.produtos.deleteMany(
  { estoque: 0 }
)

// Resultado:
// { "acknowledged" : true, "deletedCount" : 5 }
```

Retorno das operações

`acknowledged` : Confirmação da operação
`deletedCount` : Quantidade de documentos removidos

Consultas Find: Busca Básica

O Método find()

O método `find()` é a principal forma de consultar documentos em uma coleção do MongoDB. Ele retorna um cursor para os documentos que correspondem aos critérios especificados.

Sintaxe Básica

```
db.collection.find(filtro, projeção)
```

Principais Parâmetros

Parâmetro	Descrição
<code>filtro</code>	Documento com critérios de seleção
<code>projeção</code>	Campos a serem incluídos/excluídos

Exemplo Prático

```
// Consultas com método find()

// Listar todos os usuários
db.usuarios.find()

// Buscar produtos por categoria
db.produtos.find({ categoria: "Eletrônicos" })

// Buscar usuários de uma cidade específica
db.usuarios.find({ "endereco.cidade": "São Paulo" })

// Selecionar apenas nome e email dos usuários
db.usuarios.find({}, { nome: 1, email: 1 })
```

Find: Filtros e Operadores de Comparação

Operadores Principais

Para filtrar documentos no MongoDB, usamos operadores precedidos pelo símbolo `$`.

Operador	Descrição	Exemplo
<code>\$eq</code>	Igual a	<code>{ campo: { \$eq: valor } }</code>
<code>\$gt</code>	Maior que	<code>{ campo: { \$gt: valor } }</code>
<code>\$lt</code>	Menor que	<code>{ campo: { \$lt: valor } }</code>
<code>\$in</code>	Em uma lista	<code>{ campo: { \$in: [v1, v2] } }</code>

Operadores Lógicos

- `$and` Todas as condições verdadeiras
- `$or` Pelo menos uma condição verdadeira

Exemplo Prático

```
// Filtragem de produtos
// Produtos com preço entre R$ 100 e R$ 500
db.produtos.find({
  preco: { $gte: 100, $lte: 500 }
})

// Produtos em categorias específicas
db.produtos.find({
  categoria: { $in: ["Eletrônicos", "Informática"] }
})

// Produtos eletrônicos com estoque disponível
db.produtos.find({
  $and: {
    categoria: "Eletrônicos",
    estoque: { $gt: 0 }
  }
})
```

Find: Projeções e Ordenação

Refinando suas Consultas

Além de filtrar documentos, MongoDB oferece métodos para controlar quais campos exibir e em qual ordem retornar os resultados.

Projeção

Seleciona apenas os campos necessários:

```
db.collection.find(filtro, { campo1: 1, campo2: 1 })
```

Use 1 para incluir ou 0 para excluir campos

Ordenação (sort)

Ordena resultados em ordem ascendente (1) ou descendente (-1):

```
db.collection.find().sort({ campo: 1|-1 })
```

Dica

Projete apenas os campos necessários para melhorar a performance e reduzir o consumo de memória.

Exemplos Práticos

// Exemplo 1: Projeções para selecionar campos específicos

```
// Selecionando apenas nome e preço dos produtos
db.produtos.find(
  { categoria: "Eletrônicos" },
  { nome: 1, preco: 1, _id: 0 }
)

// Resultado:
// [
//   { "nome": "Smartphone Galaxy S21", "preco": 3999.90 },
//   { "nome": "Smart TV 50 4K", "preco": 2499.90 }
// ]
```

// Exemplo 2: Ordenação com sort()

```
// Produtos do mais barato para o mais caro
db.produtos.find(
  { estoque: { $gt: 0 } }
).sort({ preco: 1 }) // 1 = crescente, -1 = decrescente

// Ordenação por múltiplos campos
db.produtos.find().sort({
  categoria: 1, // Primeiro por categoria (A-Z)
  preco: -1    // Depois por preço (maior-menor)
})
```

Aggregate: Introdução e Estrutura do Pipeline

Framework de Agregação

O `aggregate()` é um poderoso framework para processamento e análise de dados no MongoDB, permitindo transformações complexas e cálculos avançados que vão além das capacidades do método `find()`.

Conceito de Pipeline

Uma sequência de estágios de processamento onde:

- Documentos fluem de um estágio para outro
- Cada estágio transforma os documentos
- A saída de um estágio é a entrada do próximo
- A ordem dos estágios impacta o resultado final



Sintaxe Básica

Estrutura do Comando

O método `aggregate()` recebe um array de objetos, onde cada objeto representa um estágio do pipeline.

```
db.collection.aggregate([
  { $stage1: { ... } },
  { $stage2: { ... } },
  { $stage3: { ... } }
])
```

// Exemplo simples de agregação

```
// Contagem de pedidos por status
db.pedidos.aggregate([
  // Agrupar por status
  {
    $group: {
      _id: "$status", // Campo para agrupamento
      total: { $sum: 1 }, // Contagem de documentos
      valorTotal: { $sum: "$valor" } // Soma do campo valor
    }
  },
  // Ordenar resultado
  {
    $sort: { total: -1 } // Ordenação decrescente
  }
])
```

Aggregate: Principais Estágios do Pipeline

\$match

Filtra documentos para processamento posterior.
Similar à sintaxe do comando `find()`.

```
{ $match: { status: "entregue", valor_total: { $gt: 100 } } }
```

\$group

Agrupar documentos por chaves específicas.
Permite aplicar operadores de agregação para cada grupo.

```
{ $group: { _id: "Categoria", total: { $sum: "$valor" } } }
```

\$project

Reformata, modifica e define campos de saída.
Permite incluir, excluir ou criar novos campos.

```
{ $project: { nome: 1, preco_ajustado: { $multiply: ["$preco", 1.1] } } }
```

\$sort

Ordena os documentos do pipeline.
Use 1 para ascendente, -1 para decrescente.

```
{ $sort: { data_pedido: -1, valor_total: -1 } }
```

\$limit / \$skip

Limita número de documentos (\$limit) ou pula documentos (\$skip).
Útil para paginação e controle de resultados.

```
{ $limit: 10 } // Limita aos primeiros 10 documentos  
{ $skip: 20 } // Pula os primeiros 20 documentos
```

\$unwind

Desconstrói arrays criando um documento para cada elemento.
Permite analisar e agregar dados dentro de arrays.

```
{ $unwind: "$itens" } // Cria um documento para cada item do array
```

\$lookup

Realiza junções entre coleções (semelhante ao JOIN SQL).
Combina documentos de múltiplas coleções.

```
{ $lookup: {  
  from: "usuarios",  
  localField: "usuario_id",  
  foreignField: "_id",  
  as: "usuario_info"  
} }
```

Outros estágios importantes

\$addFields Adiciona novos campos	\$count Conta documentos
\$replaceRoot Substitui documento	\$facet Múltiplos pipelines

Aggregate: Exemplo Prático de Pipeline

Anatomia de um Pipeline

Um pipeline de agregação processa os dados em estágios sequenciais, onde a saída de cada estágio serve como entrada para o próximo.

Como Funciona

Cada documento flui através da série de estágios, sendo transformado passo a passo até gerar o resultado final.

Passos do Pipeline do Exemplo

- 1 **\$match**
Filtra pedidos com status 'entregue'
- 2 **\$group**
Agrupar por categoria do produto e calcula total de vendas
- 3 **\$sort**
Ordena resultados por valor total de vendas (decrescente)
- 4 **\$project**
Formata os resultados para melhor apresentação

Código do Pipeline

```
// Pipeline para análise de vendas por categoria  
db.pedidos.aggregate([  
  // Estágio 1: Filtrar apenas pedidos entregues  
  { $match: { status: "entregue" } },  
  
  // Estágio 2: Desempacotar array de itens  
  { $unwind: "$itens" },  
  
  // Estágio 3: Agrupar por categoria  
  { $group: {  
    _id: "$itens.categoria",  
    totalVendas: { $sum: "$itens.valor_total" },  
    quantidadeVendida: { $sum: "$itens.quantidade" },  
    pedidos: { $addToSet: "$_id" }  
  } },  
  
  // Estágio 4: Calcular número de pedidos  
  { $project: {  
    categoria: "$_id",  
    totalVendas: 1,  
    quantidadeVendida: 1,  
    numeroPedidos: { $size: "$pedidos" }  
  } },  
  
  // Estágio 5: Ordenar por valor total  
  { $sort: { totalVendas: -1 } }  
])
```

Aggregate: Estágios \$match e \$group

Estágio \$match

Função: Filtra documentos antes de passar para o próximo estágio.
Sintaxe: Similar ao método `find()`.

```
// Sintaxe básica do $match  
{ $match: { status: "entregue", valor_total: { $gt: 100 } } }
```

Estágio \$group

Função: Agrupa documentos e aplica operações de agregação.
Sintaxe: Requer campo `_id` para agrupamento.

Principais Operadores

\$sum Soma	\$avg Média	\$min Mínimo
\$max Máximo	\$push Array	

Exemplo Prático

```
// Vendas por categoria  
db.pedidos.aggregate([  
  // Filtrar pedidos entregues  
  { $match: { status: "entregue" } },  
  
  // Agrupar por categoria  
  { $group: {  
    _id: "$categoria",  
    total_vendas: { $sum: "$valor_total" },  
    quantidade: { $sum: 1 },  
    valor_medio: { $avg: "$valor_total" },  
    maior_pedido: { $max: "$valor_total" }  
  } },  
  
  // Ordenar por total de vendas  
  { $sort: { total_vendas: -1 } }  
])  
  
// Resultado:  
// { _id: "Eletrônicos", total_vendas: 127500, quantidade: 85, ... }  
// { _id: "Roupas", total_vendas: 45600, quantidade: 120, ... }
```

Dica: Use `$match` antes de `$group` para reduzir a quantidade de dados processados e melhorar a performance.

Aggregate: Funções de Agregação

Operadores de Agregação

O MongoDB fornece vários operadores de agregação para realizar cálculos sobre conjuntos de documentos. Estes operadores são sempre usados dentro do estágio `$group`.

\$count, \$sum

`$count`: Conta documentos
`$sum`: Soma valores ou apenas 1 para contagem
{ `$sum`: "`$campo`" } ou { `$sum`: 1 }

\$avg

Calcula a média dos valores
{ `$avg`: "`$campo`" }

\$min

Retorna o menor valor
{ `$min`: "`$campo`" }

\$max

Retorna o maior valor
{ `$max`: "`$campo`" }

Dica

Funções de agregação sempre precisam de um campo para operar. Use `$sum: 1` para contar documentos quando não tiver um campo específico para somar.

Exemplo Prático

// Estatísticas de produtos por categoria

```
db.produtos.aggregate([
  { $group: {
    _id: "$categoria",
    quantidade: { $sum: 1 },           // Conta produtos
    precoMedio: { $avg: "$preco" },    // Média de preço
    precoMinimo: { $min: "$preco" },   // Preço mínimo
    precoMaximo: { $max: "$preco" },   // Preço máximo
    valorEstoque: { $sum: { $multiply: ["$preco", "$estoque"] } }
  } },
  { $sort: { quantidade: -1 } } // Ordenar por quantidade
])

// Resultado:
// { "_id": "Eletrônicos", "quantidade": 14, "precoMedio": 2145.65,
//   "precoMinimo": 89.90, "precoMaximo": 5999.90, "valorEstoque": 98500.35 }
// { "_id": "Roupas", "quantidade": 8, "precoMedio": 79.99, ... }
```

Aplicação Prática

Use estas funções de agregação para:

- Gerar relatórios consolidados
- Calcular métricas de negócio
- Análise estatística de dados
- Identificar valores extremos (outliers)

Aggregate: Análise de Pedidos por Cliente

Métricas de Cliente

O framework de agregação do MongoDB permite extrair métricas valiosas sobre o comportamento de compra dos clientes, como frequência de compra, valor total gasto e ticket médio.

Frequência de Compra

Identifica clientes mais ativos contando o número de pedidos por cliente.
`$group + $sum: 1`

Principais Aplicações

- Segmentação de clientes
- Programas de fidelidade

Exemplo Prático

// Contagem de pedidos por cliente

```
// Contagem simples de pedidos por cliente
db.pedidos.aggregate([
  { $group: {
    _id: "$usuario_id",
    totalPedidos: { $sum: 1 }
  } },
  { $sort: { totalPedidos: -1 } },
  { $limit: 5 }
])

// Resultado:
// [
//   { "_id": ObjectId("5f8e2a..."), "totalPedidos": 15 },
//   { "_id": ObjectId("5f9c3b..."), "totalPedidos": 12 },
//   { "_id": ObjectId("5f7dic..."), "totalPedidos": 8 },
//   ...
// ]
```

Aggregate: Análise Temporal de Vendas

Funções de Data no MongoDB

O MongoDB oferece diversas funções para manipulação de datas em pipelines de agregação, permitindo análises temporais sofisticadas.

Principais Operadores de Data

- `$year`: Extrai o ano (2023)
- `$month`: Extrai o mês (1-12)
- `$dayOfMonth`: Extrai o dia (1-31)
- `$dayOfWeek`: Dia da semana (1=domingo, 7=sábado)
- `$week`: Semana do ano (0-53)

Aplicações em Business Intelligence:

- Vendas por mês/trimestre/ano
- Comparação de períodos (crescimento)
- Sazonalidade de produtos
- Tendências de compra por dia da semana
- Relatórios de performance temporal

Exemplos Práticos

// Análise de vendas por mês

```
db.pedidos.aggregate([
  // Filtrar apenas pedidos concluídos
  { $match: {
    status: "entregue",
    data: { $gte: new Date("2023-01-01") }
  } },
  // Agrupar por mês e ano
  { $group: {
    _id: {
      mes: { $month: "$data" },
      ano: { $year: "$data" }
    },
    total_vendas: { $sum: "$valor_total" },
    quantidade_pedidos: { $sum: 1 },
    ticket_medio: { $avg: "$valor_total" }
  } },
  // Ordenar por ano e mês
  { $sort: { "_id.ano": 1, "_id.mes": 1 } }
])

// Resultado:
// { "_id": { "mes": 1, "ano": 2023 }, "total_vendas": 28750, "quantidade_pedidos": 1200, "ticket_medio": 23.96 }
// { "_id": { "mes": 2, "ano": 2023 }, "total_vendas": 31200, "quantidade_pedidos": 1300, "ticket_medio": 23.96 }
```

Aggregate: \$lookup para Junções

O que é \$lookup?

O operador `$lookup` realiza uma junção à esquerda (left outer join) entre duas coleções no MongoDB, permitindo combinar dados relacionados.

Sintaxe Básica

```
{
  $lookup: {
    from: "coleção_destino",
    localField: "campo_local",
    foreignField: "campo_destino",
    as: "nome_resultado"
  }
}
```

pedidos → usuarios

Importante: O resultado do `$lookup` é sempre um array, mesmo quando há apenas um documento correspondente.

Exemplo Prático

```
// Junção entre pedidos e usuários
db.pedidos.aggregate([
  // Estágio de junção
  {
    $lookup: {
      from: "usuarios", // Coleção a ser conectada
      localField: "usuario_id", // Campo na coleção pedidos
      foreignField: "_id", // Campo na coleção usuarios
      as: "usuario" // Nome do array resultado
    }
  },
  // Estágio de projeção para formatar a saída
  {
    $project: {
      _id: 1,
      data: 1,
      valor_total: 1,
      status: 1,
      "usuario.nome": 1,
      "usuario.email": 1
    }
  }
])
```

Aggregate: \$unwind

O que é \$unwind?

O operador `$unwind` "desempacota" arrays dentro de documentos, criando um novo documento para **cada elemento** do array.

Funcionamento

Dado um documento com um campo array:

```
{ pedido: 1234, itens: [A, B, C] }
```

Após `$unwind`: "`$itens`":

```
{ pedido: 1234, itens: A }
```

```
{ pedido: 1234, itens: B }
```

```
{ pedido: 1234, itens: C }
```

Exemplo Prático

```
// Itens de pedidos
// Documento de pedido
{
  "_id": ObjectId("123abc"),
  "cliente": "Maria",
  "data": ISODate("2023-06-15"),
  "itens": [
    { "produto": "Notebook", "valor": 3500 },
    { "produto": "Mouse", "valor": 120 }
  ]
}

// Usando $unwind para analisar cada item
db.pedidos.aggregate([
  { $unwind: "$itens" }
])

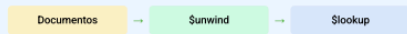
// Resultado:
// Documento 1:
{
  "_id": ObjectId("123abc"),
  "cliente": "Maria",
  "data": ISODate("2023-06-15"),
  "itens": { "produto": "Notebook", "valor": 3500 }
}
```

\$unwind Avançado: Análise Complexa de Itens

Combinando Operadores

O verdadeiro poder do `$unwind` aparece quando combinamos com outros operadores:

Pipeline Típico



- 1 `$unwind` - Explode os arrays
- 2 `$lookup` - Conecta com outras coleções

Principais Casos de Uso:

- Análise de popularidade de produtos
- Estatísticas de vendas por categoria
- Identificação de padrões de compra

Exemplo Prático

```
// Análise de produtos mais vendidos
db.pedidos.aggregate([
  // Filtrar apenas pedidos entregues
  { $match: { status: "entregue" } },

  // Explode o array de itens (um documento por item)
  { $unwind: "$itens" },

  // Conecta com dados do produto
  { $lookup: {
    from: "produtos",
    localField: "itens.produto_id",
    foreignField: "_id",
    as: "produto_detalhes"
  }},

  // Descompacta os detalhes do produto
  { $unwind: "$produto_detalhes" },

  // Agrupa por produto com métricas
  { $group: {
    _id: "$produto_detalhes._id",
    nome_produto: { $first: "$produto_detalhes.nome" },
    categoria: { $first: "$produto_detalhes.categoria" },
    quantidade_total: { $sum: "$itens.quantidade" },
    valor_total: { $sum: {
      $multiply: ["$itens.preco_unitario", "$itens.quantidade"]
    } }
  }},

  // Ordena por valor total decrescente
  { $sort: { valor_total: -1 } },

  // Limita a 5 produtos mais vendidos
  { $limit: 5 }
])
```